LCM 无损上下文管理

> **来源**：本地 PDF《LCM（Lossless Context Management） 无损上下文管理》（Microsoft Print to PDF，6 页，无文字层）经 OCR 转写为正文；**示意图无法可靠 OCR 处保留原图**。微信公众号：https://mp.weixin.qq.com/s/hx1ZKIclwRgZ1Nrcjr8SUA
> **可复制全文 PDF**（便于离线检索）：/files/lcm/lcm-lossless-context-full-text.pdf（仓库内由 `python3 tooling/build_lcm_pdf.py` 自本文生成，需已安装 `pandoc` 与 `wkhtmltopdf`）

摘要

LCM（Lossless Context Management）是一种**确定性的**大语言模型**上下文管理**架构。核心思想是：当对话**超出**模型上下文窗口时，**不丢弃**任何消息，而是通过**分层摘要 DAG** 压缩历史，同时保留**全部原始内容**在存储中的**可检索性**（文中以 **SQLite** 为例），从而在窗口内「看起来变短了」，但底层数据**仍可无损展开**。

传统方式及其问题

在传统设置里，对话保存在固定的 **token budget**（模型一次能容纳的最大文本量）中；每条消息都会占用预算。该预算不仅是硬性上限——往往还要**预留余量**，以便在需要时让模型再生成摘要。

当上下文用量超过某个**阈值**（文中示例约为预算的 **80%** 左右），系统会触发**摘要调用**：把尽可能多的消息**批量**送给模型做摘要；这一过程常常需要约 **1～2 分钟**。随后，这些内容常被**合并**为**单一的扁平摘要**，窗口空间被腾出，但**细粒度信息随之丢失**。扁平摘要难以保留长对话的完整结构，模型可能**记错细节**、与先前决策**矛盾**，或重复追问已经交代过的信息；并且这一过程往往**难以回溯**到压缩前的原文面貌。

传统方式：超阈值后批量摘要导致细节不可恢复（文中原图）

图示要点（与正文互补）：传统路径可概括为「超阈值 → 批量摘要 → 细节永久丢失」；对比 token 条与「单一摘要」的取舍。

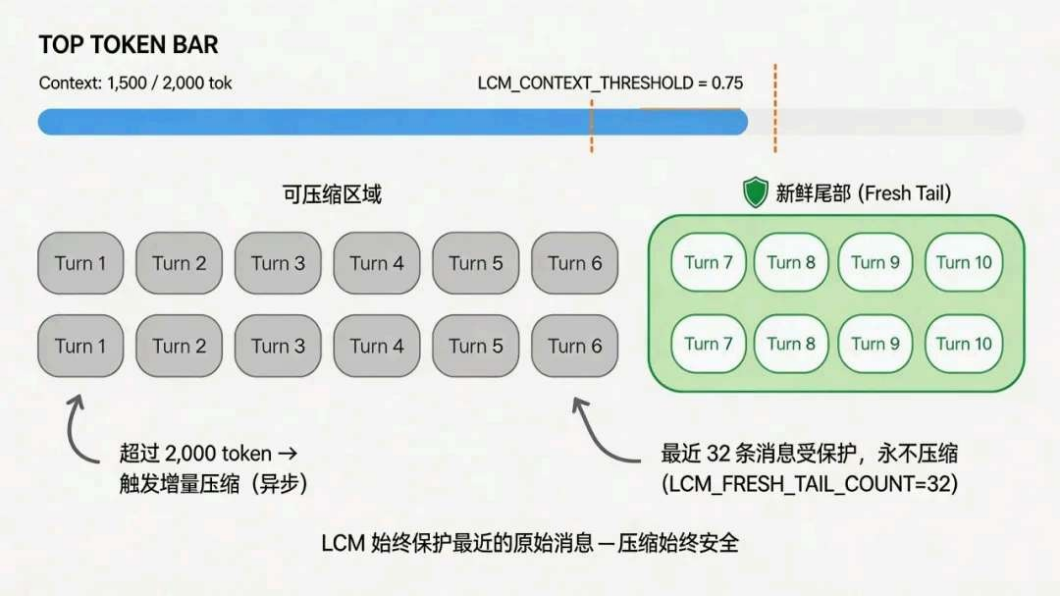
LCM 的方式：分层压缩 + 不丢源数据

LCM 与传统方式的**起点相同**——对话持续追加消息、token 持续累积；差异在于：接近预算时**不是简单截断**，而是把更早的消息组织为**分层摘要**，并在结构中**保留**对**源消息**的引用，使得任意被摘要块在需要时仍可按图索骥展开。

新鲜尾部（Fresh Tail）

LCM **始终保护**最近的若干条**原始消息**，即文中的「**新鲜尾部**」；这些消息**永不**被压缩。更早的消息则可以进入摘要流程。文中示例：**最近 32 条**受保护（参数可记为 `LCM_FRESH_TAIL_COUNT=32`），并通过 **TOP TOKEN BAR** 一类可视化方式持续监控已用 token / 预算（示例阈值如 `LCM_CONTEXT_THRESHOLD = 0.75`）。

当「新鲜尾部」**之外**仍保留的原始消息合计超过某一规模（文中有 **2000 token** 量级的触发条件描述）时，LCM 触发**增量压缩**；该过程通常是**异步**的，**不打断**当前对话轮次。由于**源消息持久化**在数据库中（如 SQLite），压缩在当时被描述为「**安全**」的——压缩的是**呈现层/摘要节点**，而不是物理删除原始记录。



新鲜尾部、阈值条与「超 2000 token 触发异步增量压缩」示意（文中原图）

增量压缩（深度 0）

在增量压缩中，**满足条件的消息块**会与**结构化压缩提示词**一并发送给模型。生成的摘要会**替换**对话中对应片段的「展开前」占位，但对**源消息**的**引用/链接**仍保留在结构里，以便后续按需展开。

文中给出的**深度 0**提示词意图包括：为**后续对话轮次**总结片段；保留**决策、理由、约束、进行中的任务**；去除重复与寒暄；结尾可注明类似「展开可查看详情：〈被压缩内容〉」的提示，给后续检索或展开留钩子。

增量压缩：消息块 + 结构化提示词 → LLM → 深度 0 摘要节点（文中原图）

图中对应关系（意译）：符合条件的原文块经深度 0 提示词生成摘要节点；源消息仍驻留在 SQLite，对话继续在紧凑上下文中进行。

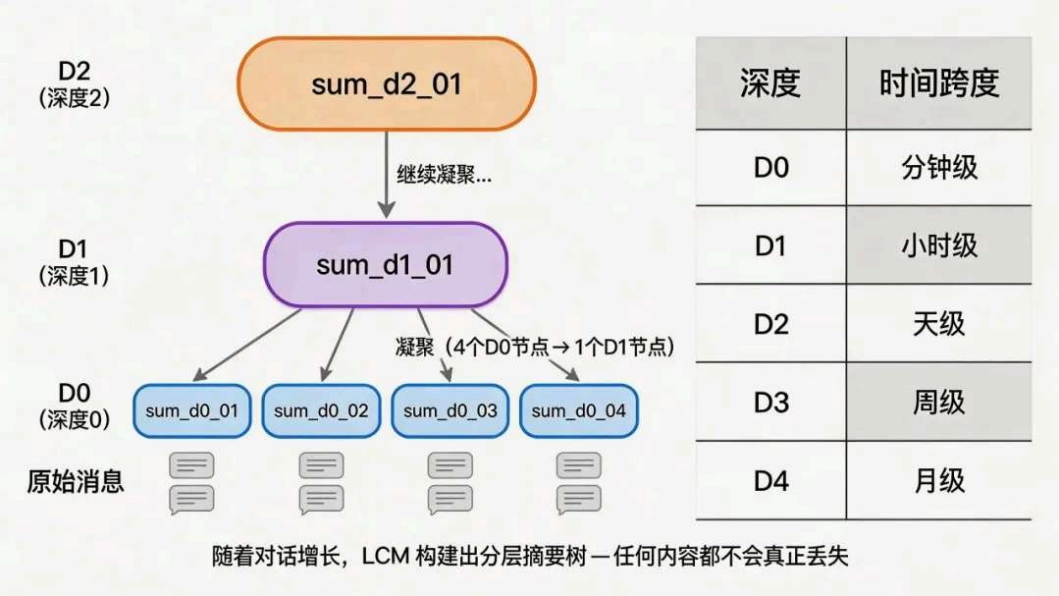
凝聚（Condensation）与多深度摘要树

当同一**深度**积累了足够多的摘要节点后，LCM 会触发**凝聚（condensation）**：文中示例为同一深度集齐 **4** 个节点后，合并为更高一层的节点（如由多个深度 0 摘要**凝聚**为深度 1 节点）。更高层使用**不同的**结构化提示词：例如深度 1 的输入是「子摘要」而非原始消息，强调**决策/结果/阻塞点/进行中状态**，并引入带时间戳的**关键事件时间线**等。

随着对话变长，LCM 形成**多深度**的摘要结构（文中以树/DAG 描述），典型区分是：

- **深度 0**：偏**具体**——决策、理由、精确技术细节。
- **深度 1**：偏**脉络**——演变过程、结果、当前状态。
- **深度 2**：偏**长期叙事**——仍在生效的决策与里程碑时间线；在文风口径中可在**数周**尺度上仍保持可用。

文中还以**分钟 / 小时 / 天 / 月**等意象，与各深度可能对应的**时间跨度**做对照，帮助理解「越深、越提炼、覆盖时间越长」。



凝聚与分层：多深度节点与时间跨度示意（文中原图）

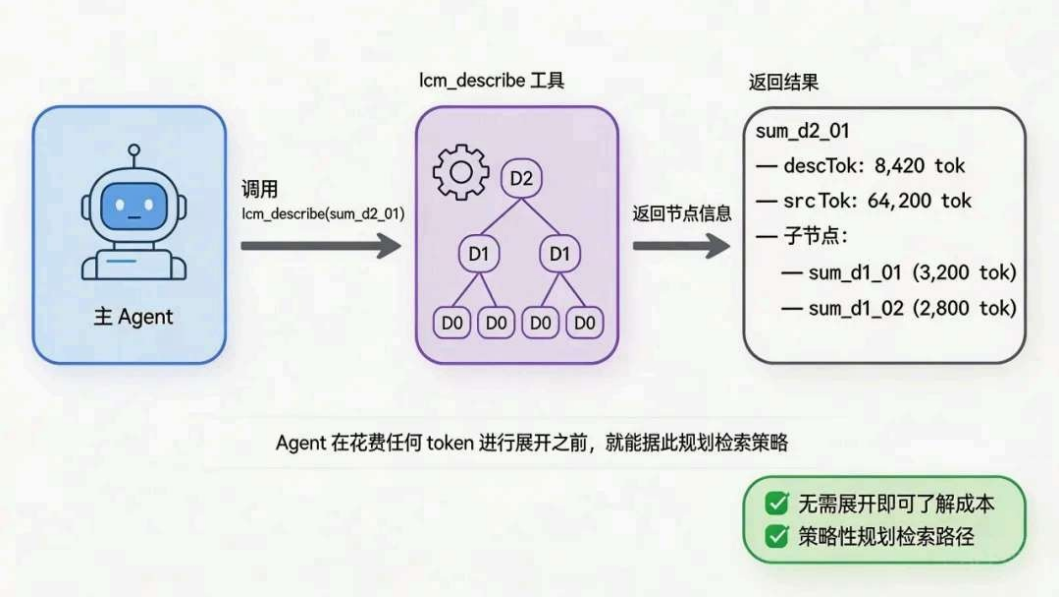
在理想叙述中：对话体量已经很大，但**送进模型的主上下文**仍可保持紧凑；**没有任何消息被从持久存储中删掉**——它们仍活在 **DAG** 的引用与展开路径里。

主 Agent 如何读 DAG：有界检索与子 Agent

一个**深度 2**的摘要背后，可能对应**数万 token**的原始消息；若把这些一股脑塞回**主 Agent**上下文，会立刻再次触发压缩。因此 LCM 采用**主-子 Agent**分工：子 Agent 在**隔离**的上下文里沿 **DAG** 导航，可在例如 **~20k token** 的子树上工作，但通过**策略性展开**（先评估再展开最相关节点），使实际占用的 token budget 可能仅**数百～一千**量级，从而**不撑爆**主 Agent 窗口。

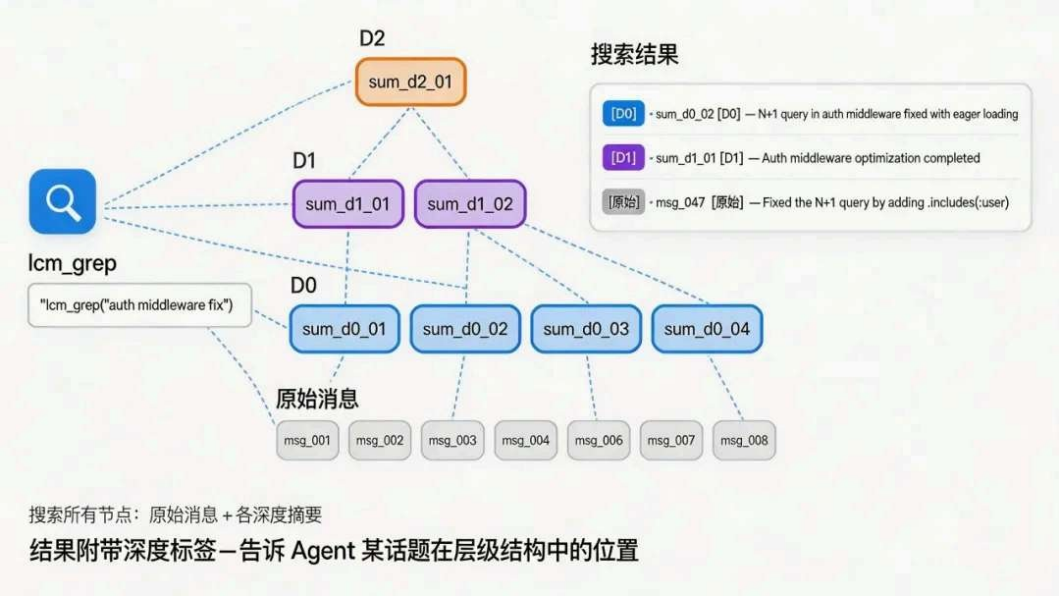
文中工具包括（名称以原文为准）：

工具	作用（据原文归纳）
`lcm_describe`	在不实际展开正文前，返回子树的 **descTok**（摘要侧规模）、**srcTok**（源消息规模）及**子节点清单**，用于规划检索策略。
`lcm_grep`	在 DAG **各层节点**（原始消息与各深度摘要）中搜索；结果带**深度标签**，提示命中处处于结构的哪一层。
`lcm_expand` / `lcm_expand_query`	需要更多细节时，通过**委托授权（delegation grant）**等机制，生成**子 Agent**在预算内**策略性遍历** DAG。



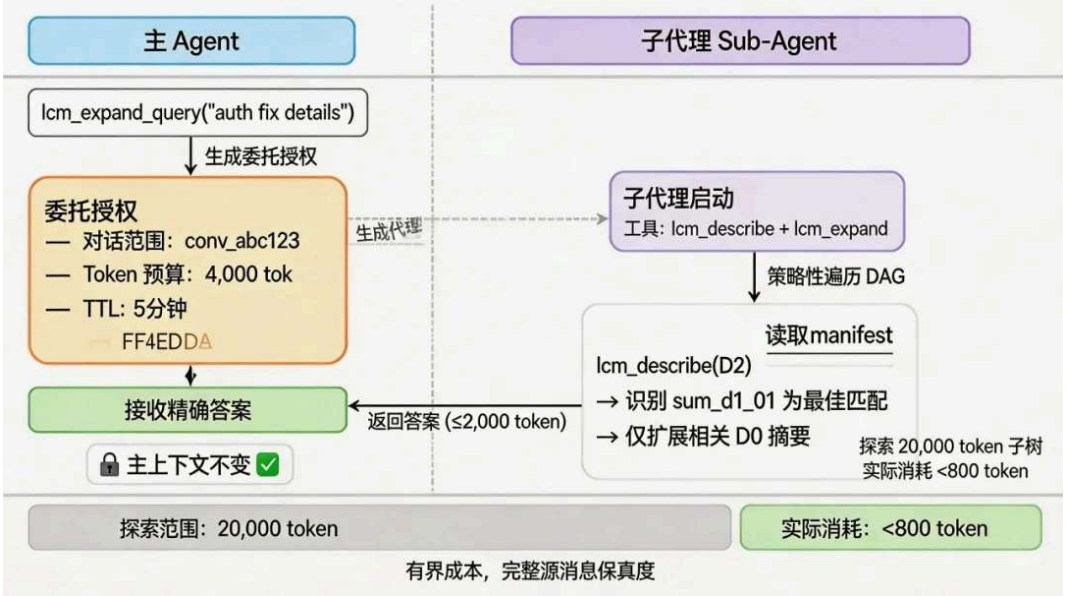
lcm_describe：调用与返回字段示意（文中原图）

子 Agent 获得如 `lcm_describe`、`lcm_expand` 及有限 **token 预算**（文中示例 **4000 tok**）与 **TTL**；它会先读结构、再选择最匹配的子树展开。文中给出典型数字：在**约 20 000 token**的探索范围下，通过有界操作，实际消耗可低于 **800 token**，主上下文不变，却仍可取回**很久之前轮次**的精确细节。



子 Agent：有界预算下策略性遍历 DAG（文中原图）

（`lcm_grep` 的界面化示意在原文中为独立配图，版式与表格难以 100% OCR；若需像素级对照请查阅 PDF 或下节 PDF 归档。）



子代理有界展开：探索范围 vs 实际消耗对比（文中原图）

结语（原文风向）

在上述叙事下，LCM 试图在「**必须控制上下文长度**」与「**不能丢可追溯细节**」之间取折中：**大脑**仍是 LLM，**骨架与记忆**则由分层 DAG、持久源消息与工具化检索共同承担。

参考链接

1. 论文/技术页：https://papers.voltropy.com/LCM
2. 开源实现：https://github.com/martian-engineering/lossless-claw
3. 站点：https://losslesscontext.ai